Digital Assignment 1

Software Engineering

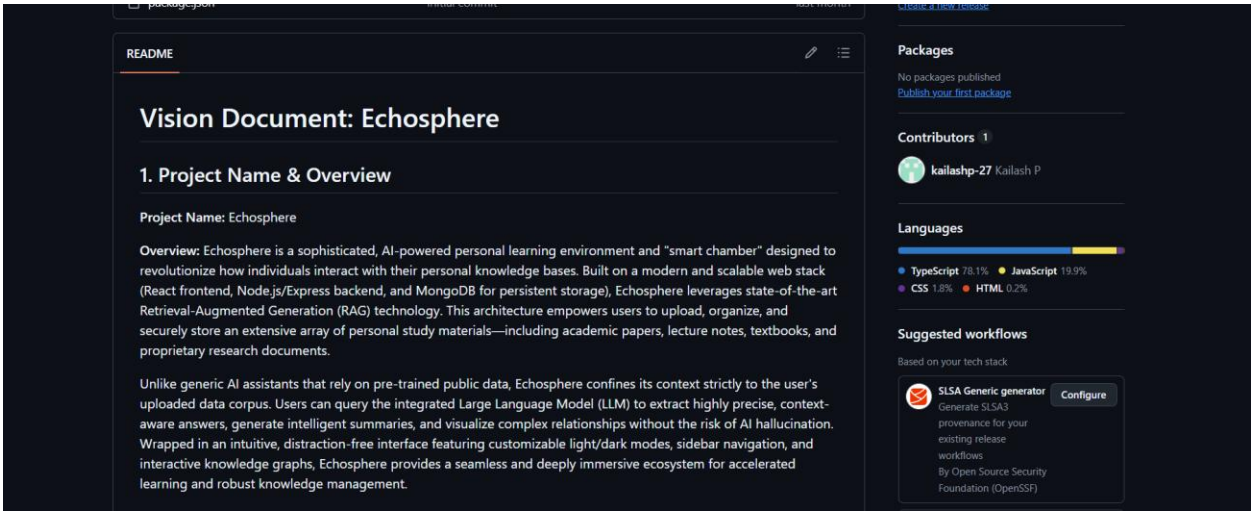
Name: KAILASH P

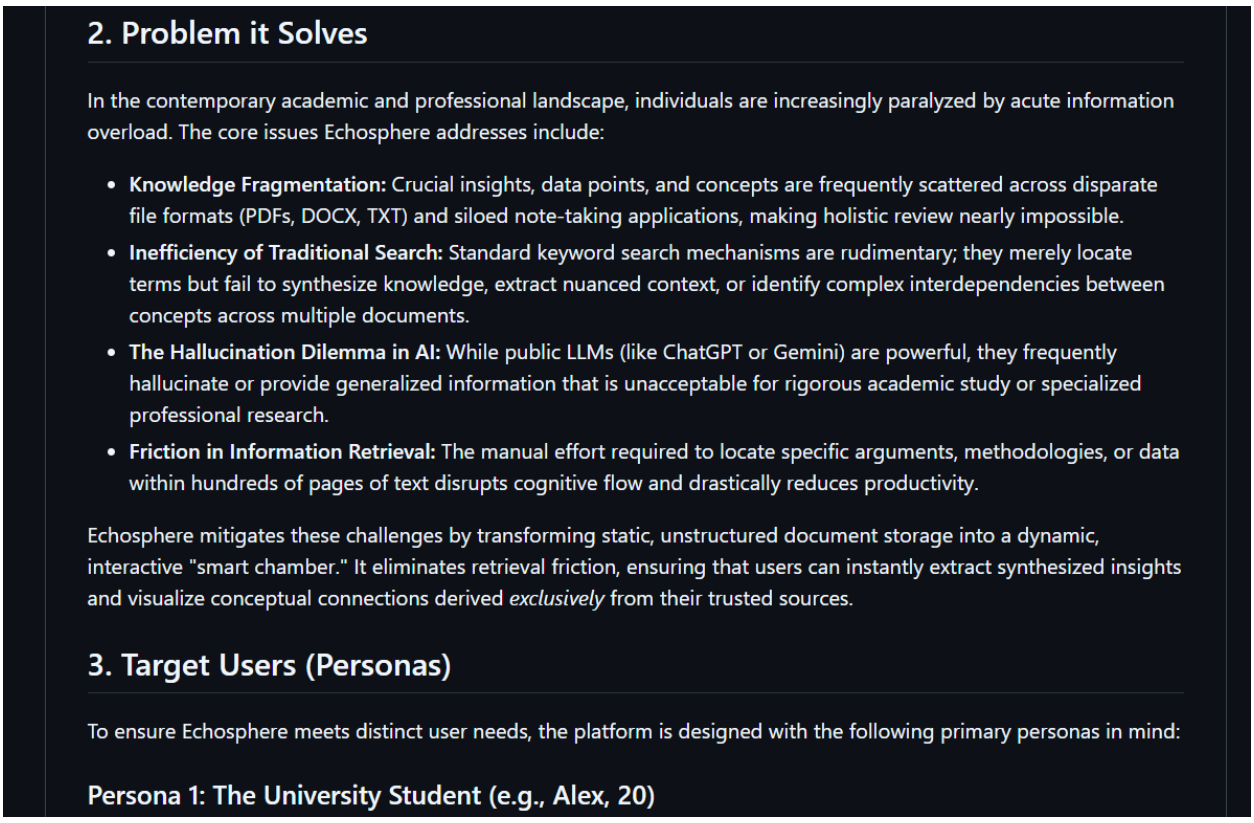
Register No: 24BRS1382

Repo link: https://github.com/kailashp-27/EchoSphere

Vision document:

Readme repo link- https://github.com/kailashp-27/EchoSphere/blob/main/README.md





- **Demographic & Background:** A junior university student double-majoring in Computer Science and Cognitive Psychology, handling a massive volume of dense coursework.
- **Pain Points:** Struggles to synthesize hundreds of pages of lecture slides, empirical research papers, and textbook chapters before midterms and finals. Alex frequently forgets where specific psychological theories were mapped to computational models across different semesters.
- **Goals & Use Case:** Alex needs a centralized hub to upload course syllabi, PDFs, and typed notes. Before a critical exam, Alex uses Echosphere to query, "Synthesize the relationship between neural networks and human cognitive memory models based on my Week 4 and Week 7 notes." Furthermore, Alex utilizes the knowledge graph to visually map dependencies between syllabus topics, drastically reducing study time and improving retention.

Persona 2: The Professional Researcher (e.g., Dr. Elena, 35)

- **Demographic & Background:** A postdoctoral researcher in Environmental Science currently conducting a comprehensive literature review for a meta-analysis on climate change mitigation strategies.
- **Pain Points:** Manages an unwieldy, constantly growing library of complex academic papers. Dr. Elena needs to meticulously cross-reference specific data points, statistical methodologies, and conclusions without the risk of AI tools fabricating information or pulling from irrelevant, non-peer-reviewed web sources.
- **Goals & Use Case:** Dr. Elena curates and uploads her localized library of PDFs to Echosphere. She relies on the system's strict RAG implementation to execute highly specific queries (e.g., "Extract the confidence intervals and sample sizes used in the carbon sequestration methodologies across all 2023 papers"). She trusts the output because Echosphere cites the exact documents and passages it used to generate the response.

4. Vision Statement

To empower lifelong learners and rigorous researchers by transforming static, fragmented data into an interactive, strictly contextualized engine of rapid knowledge discovery and profound synthesis.

5. Key Features / Goals

Echosphere's architecture is driven by the following core features and technical objectives:

- **Strict RAG-Based Querying Engine:** An intelligent conversational interface that utilizes Retrieval-Augmented Generation to answer complex, multi-faceted queries strictly based on the user's uploaded document corpus, virtually eliminating external AI hallucinations.
- **Comprehensive Document Ingestion & Parsing:** A robust backend system allowing users to upload, organize in folders, and permanently store various file formats (PDFs, DOCX, raw text). It utilizes optimized parsing libraries (`pdf-parse` , `mammoth`) to accurately extract and sanitize text.
- **Interactive Knowledge Graph Visualization:** Automated generation of dynamic, 2D physics-based network graphs (using `react-force-graph`). This tool visually maps out core concepts (Nodes) and their relationships/dependencies (Edges) automatically extracted from the selected documents.
- **Customizable "Smart Chamber" UI/UX:** A sleek, highly responsive React frontend designed for deep focus. It features intuitive sidebar navigation, configurable light/dark enterprise themes (via Tailwind CSS), and isolated workspaces for different AI tasks.
- **Modular AI Tools (Summarizer & Explainer):**
  - *Smart Summarizer:* Condenses large, unwieldy documents into digestible formats, configurable by length (bullet points, medium overview, detailed report).
  - *Concept Explainer:* Employs the Feynman Technique, dynamically adapting explanations of complex topics to the user's chosen depth (e.g., beginner-friendly, child-like simplicity, or expert-level technical breakdown).
- **Secure & Dynamic Prompt Management:** A dedicated workspace to save, iterate, and manage custom AI prompts, coupled with dynamic API key configuration stored securely in the browser's local storage.

6. Success Metrics

To objectively evaluate Echosphere's success and operational efficiency, the following quantitative and qualitative metrics are established:

- **Query Performance & Latency:** 95% of RAG-based queries must return accurate, fully contextualized responses within 3 to 5 seconds, ensuring a conversational and frictionless user experience.
- **Ingestion & Parsing Reliability:** Achieve a 99% success rate for parsing and vectorizing supported file types (PDF, DOCX) without significant data loss, text garbling, or formatting corruption.
- **User Retention & Engagement:** Attain a 60% Weekly Active User (WAU) retention rate among early adopters, with users executing an average of 10+ intelligent queries per active session.
- **Hallucination Rate (Fidelity Metric):** Maintain a near-zero (<1%) instance of AI hallucinations. This is measured via internal testing and user feedback mechanisms (e.g., upvote/downvote on RAG-generated answers for accuracy against the source text).
- **System Uptime:** Ensure 99.9% availability of the frontend interface and backend API services during peak usage hours.

7. Assumptions & Constraints

Technical & Operational Constraints

- **Third-Party LLM Dependency:** The core cognitive intelligence relies heavily on external LLM APIs (e.g., Google Gemini). The system is therefore subject to the provider's API rate limits, network latency, unpredictable downtime, and changing pricing structures.
- **Computational & Storage Overhead:** Processing, chunking, and embedding massive volumes of text (especially long-form academic textbooks) require significant backend memory and database storage (MongoDB). This may incur computational bottlenecks during the initial document ingestion phase.
- **Complex Document Parsing:** Extracting clean text from highly formatted PDFs (e.g., multi-column academic papers featuring complex mathematical formulas, embedded images, and intricate tables) is notoriously difficult and may result in imperfect data chunking.

Project Assumptions

- **User Technical Literacy:** It is assumed that target users possess a baseline understanding of digital file management (uploading/organizing files) and are comfortable interacting with conversational AI interfaces.
- **API Key Provision:** It is assumed that users will provide their own valid LLM API keys (e.g., Gemini) to power the application, shifting the API cost burden to the user in a localized deployment model.
- **Data Privacy Acceptance:** Users are willing to upload their personal, potentially sensitive study materials to the database with the understanding that the text will be processed via external LLM APIs for the purpose of generating responses.

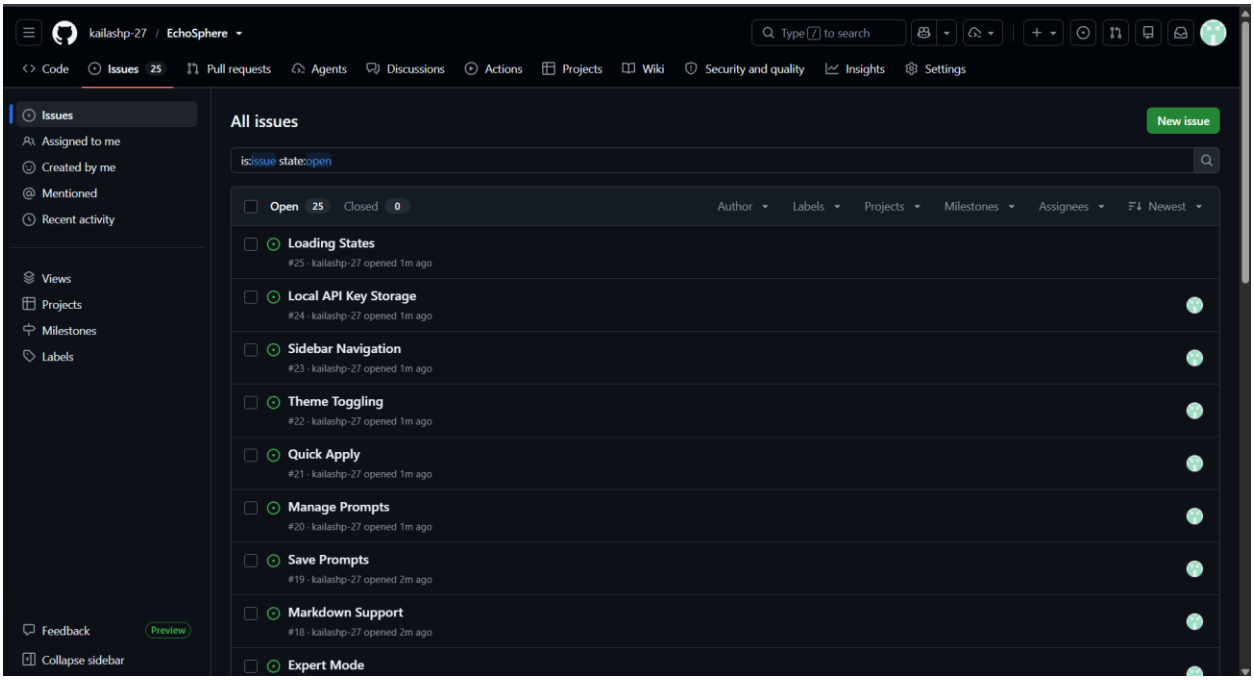
🌿 Branching Strategy (GitHub Flow)

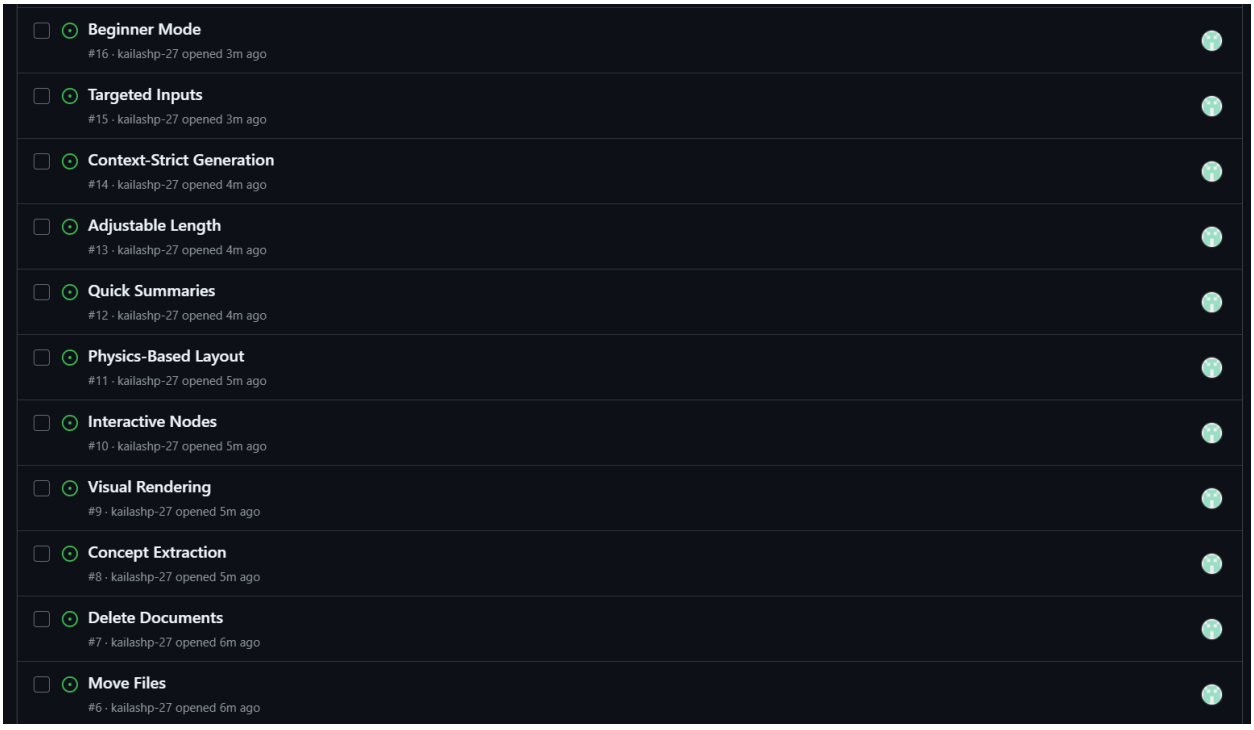
For EchoSphere, we strictly adhere to the **GitHub Flow** branching strategy to maintain a clean, organized, and stable repository during development.

1. `main` **Branch:** This is the primary, production-ready branch. All code in `main` is stable and functional. Direct commits to `main` are avoided to prevent breaking the core application.
2. **Feature Branches:** For every new module (e.g., Knowledge Graph, Concept Explainer), a dedicated branch is created from `main`. We use descriptive naming conventions such as:
   - `feature/pdf-ingestion`
   - `feature/knowledge-graph-ui`
   - `bugfix/gemini-api-key-error`
3. **Pull Requests (PRs):** Once a feature is complete locally, a Pull Request is opened against the `main` branch. This allows for code review and automated checks before integration.
4. **Merge & Delete:** After a successful review and merge into `main`, the feature branch is deleted to keep the repository uncluttered.

2. User Stories 25 user stories tracked as GitHub Issues on our Project board

Issue link- https://github.com/kailashp-27/EchoSphere/issues







3. MoSCoW and Figma Wireframes

MoSCoW prioritization:

- Must Have
- Should Have
- Could Have
- Won't Have

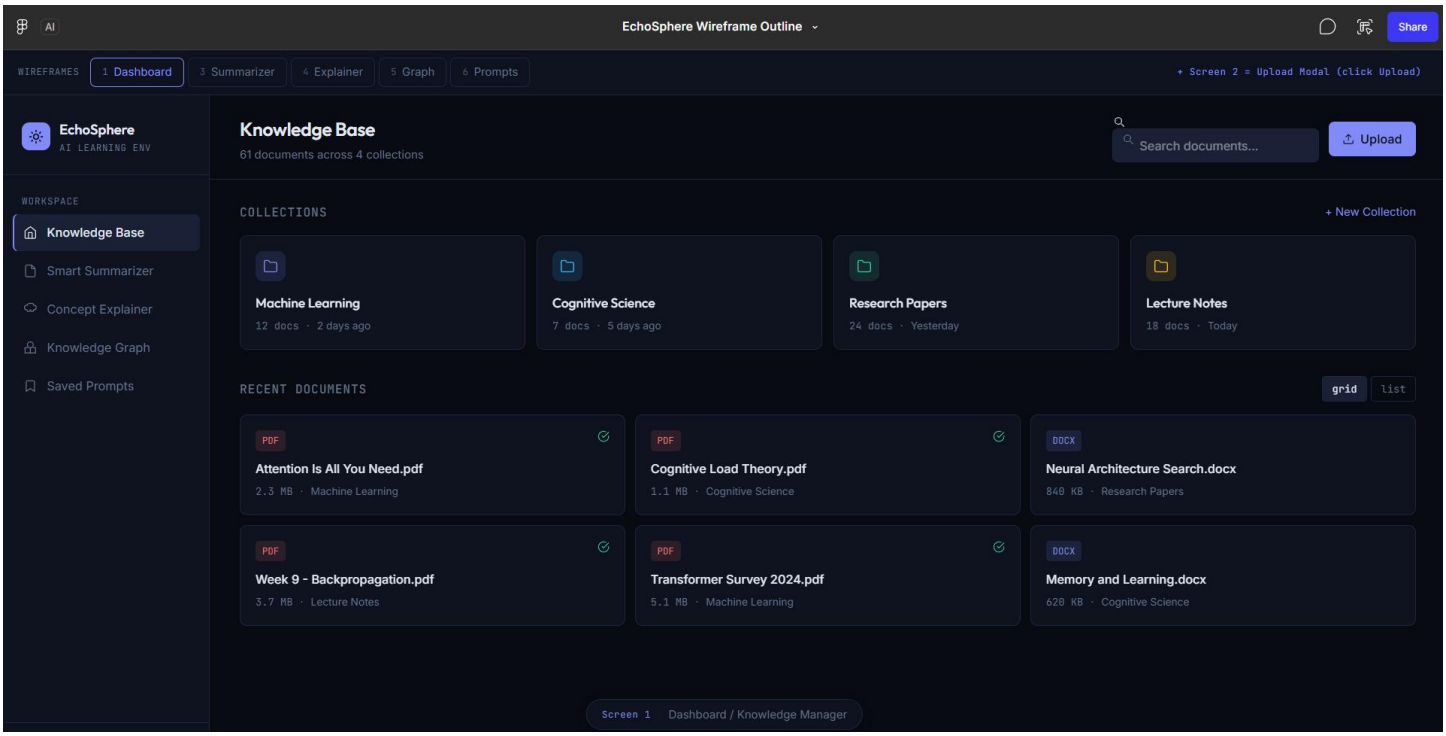
Priority	User Story	Feature Area	Justification
Must	Upload and ingest PDF/DOCX files	Knowledge Manager	Nothing works without study materials. This is the foundational data source for the RAG pipeline.
Must	Parse and extract clean text (Backend)	Backend Processing	LLMs cannot read raw files directly; extracting text buffers is a mandatory step for AI processing.
Must	Secure local API key storage	Authentication	Required to connect to Gemini AI without building a complex multi-user backend authentication system.
Must	Generate basic document summaries	AI Tools	The core minimum viable feature to prove the LLM integration and RAG pipeline are functioning correctly.

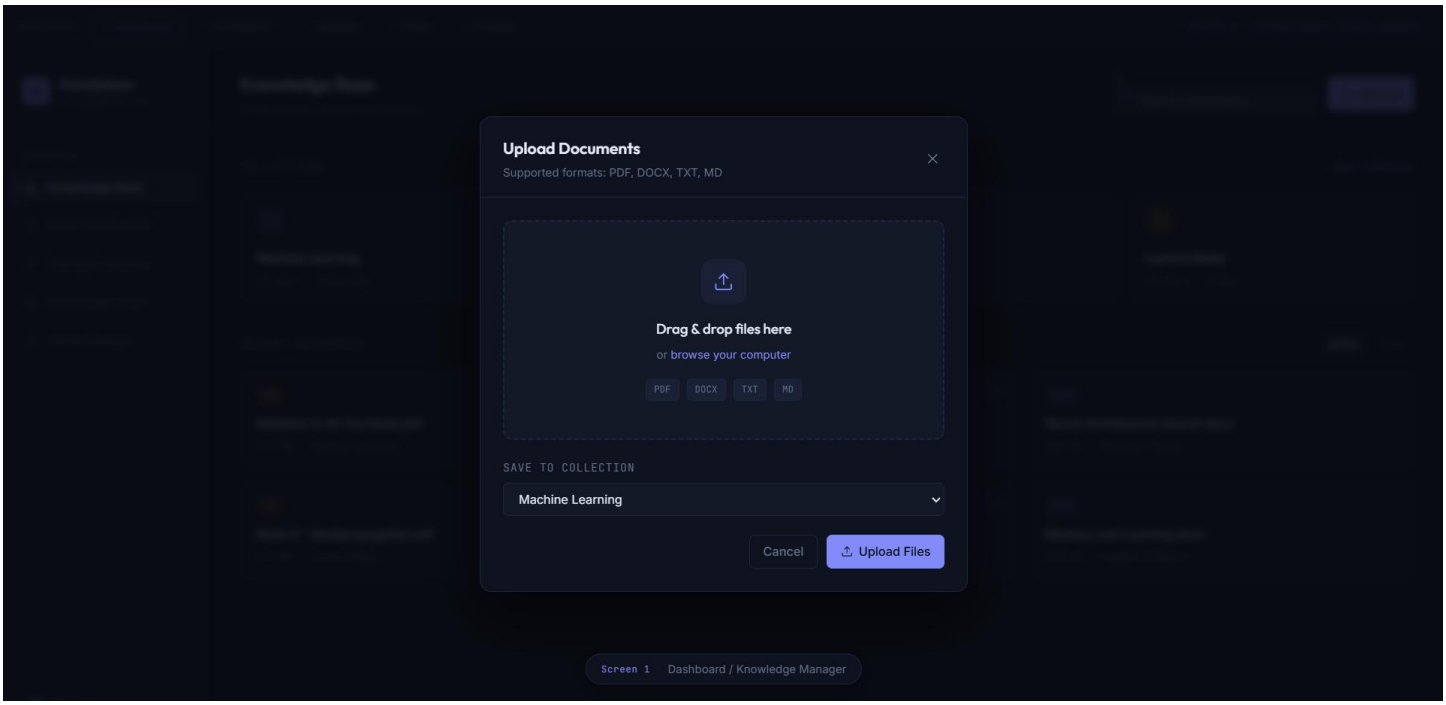
Priority	User Story	Feature Area	Justification
Must	Store and retrieve notes (MongoDB)	Database	Essential for persisting user knowledge across sessions; without this, it is just a stateless demo.
Should	Organize documents into folders	Knowledge Manager	Important for usability as the user's study materials grow, but a flat list works for an initial prototype.
Should	Extract nodes/edges for Knowledge Graph	AI Pipeline	Sets up the core differentiator of the app (visual learning), but the app still functions as a text-based study tool without it.
Should	Adjustable output lengths (Feynman/Summary)	AI Tools	Highly valuable for tailored learning, but fixed-length AI responses are sufficient for basic functionality.
Should	Persistent sidebar navigation	UI/UX	Major usability win for switching between AI tools, but basic page routing works as a fallback.
Could	Interactive 2D Knowledge Graph UI	Data Visualization	A strong visual feature utilizing react-force-graph, but complex to render; considered an enhancement over core text tools.
Could	Saved custom AI prompts workspace	AI Tools	Great for power users to speed up workflows, but manual prompt entry works as a functional fallback.
Could	Enterprise Dark Mode toggle	UI/UX	Significantly improves the reading experience for students, but is strictly an aesthetic enhancement.

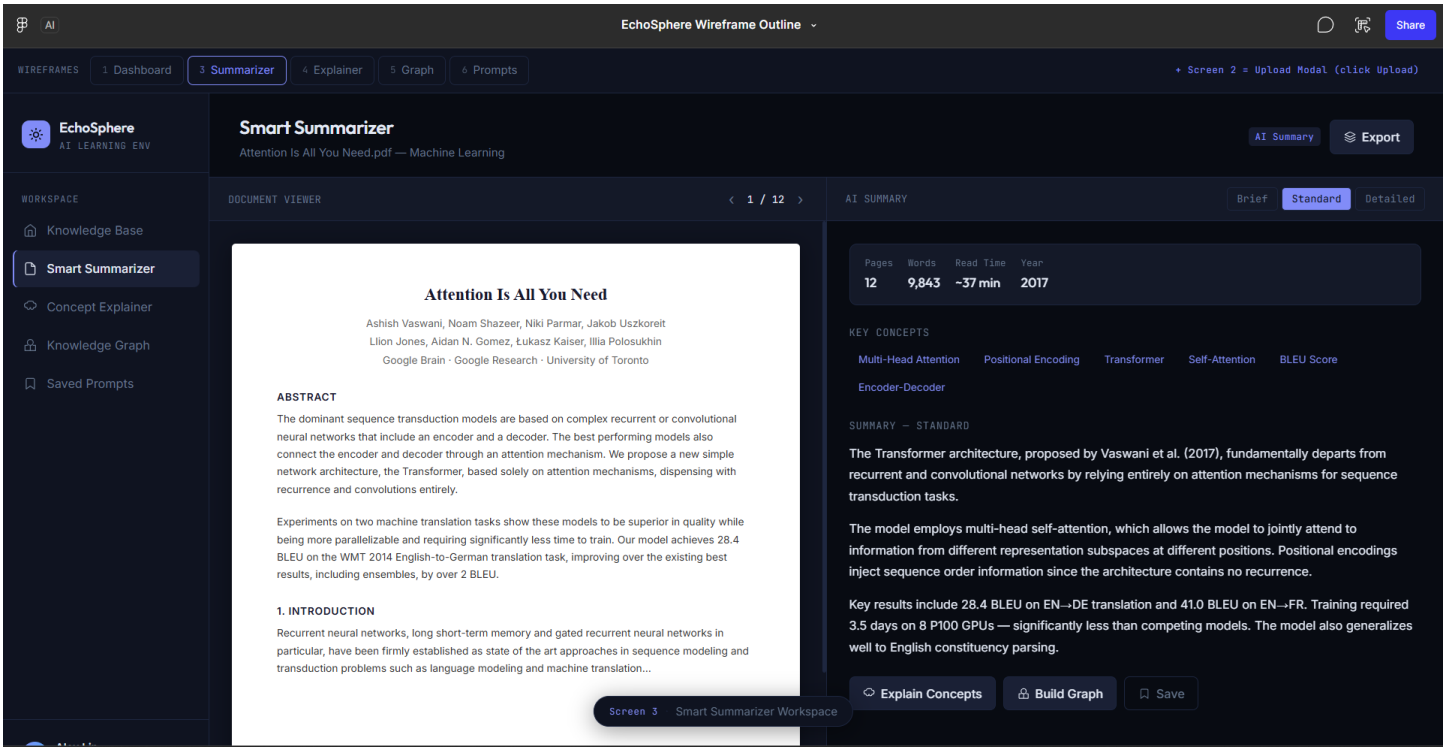
Priority	User Story	Feature Area	Justification
Won't	Multi-user JWT login system	Infrastructure	Out of scope for this specific assignment release; focusing on a single-user local environment first.
Won't	Image OCR for scanned documents	Knowledge Manager	Too complex for the current timeline; development must focus strictly on text-selectable PDFs and DOCX files.

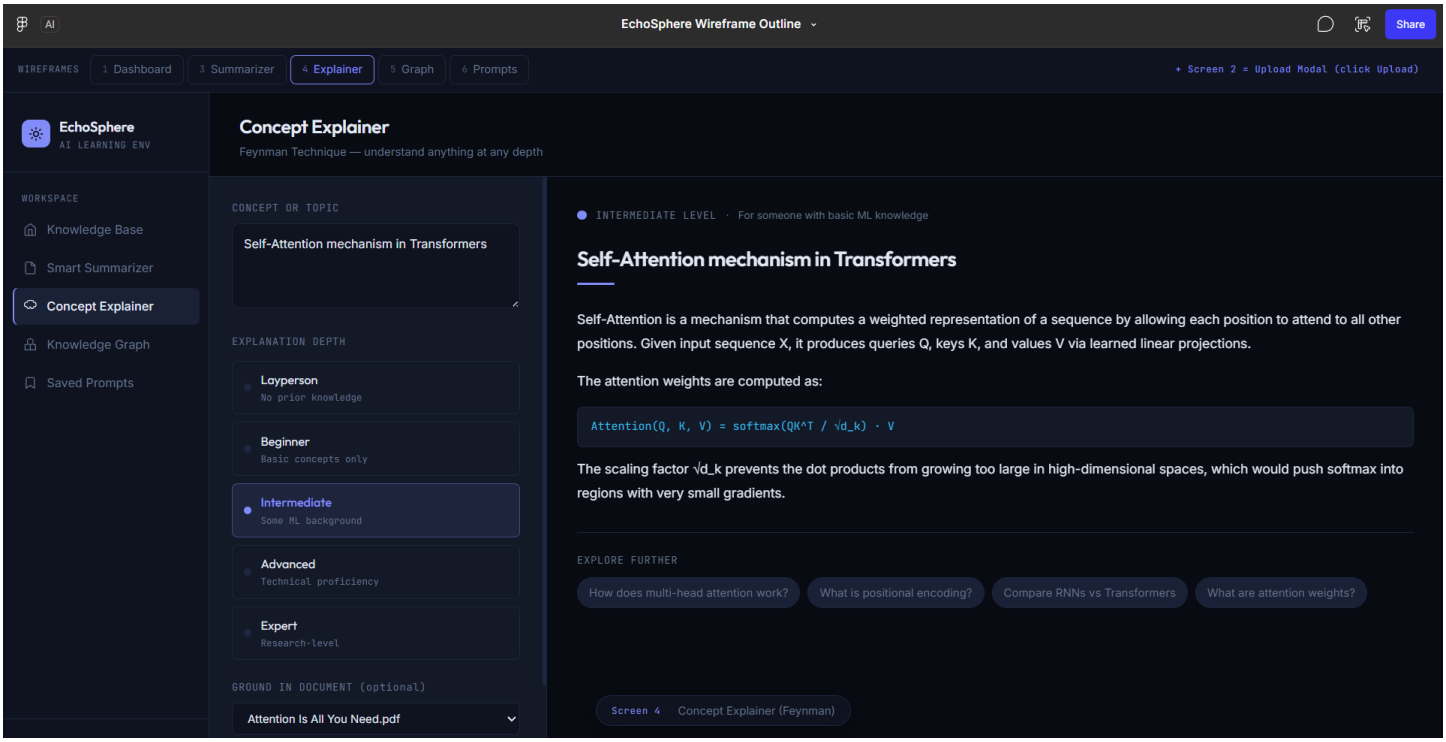
Figma –

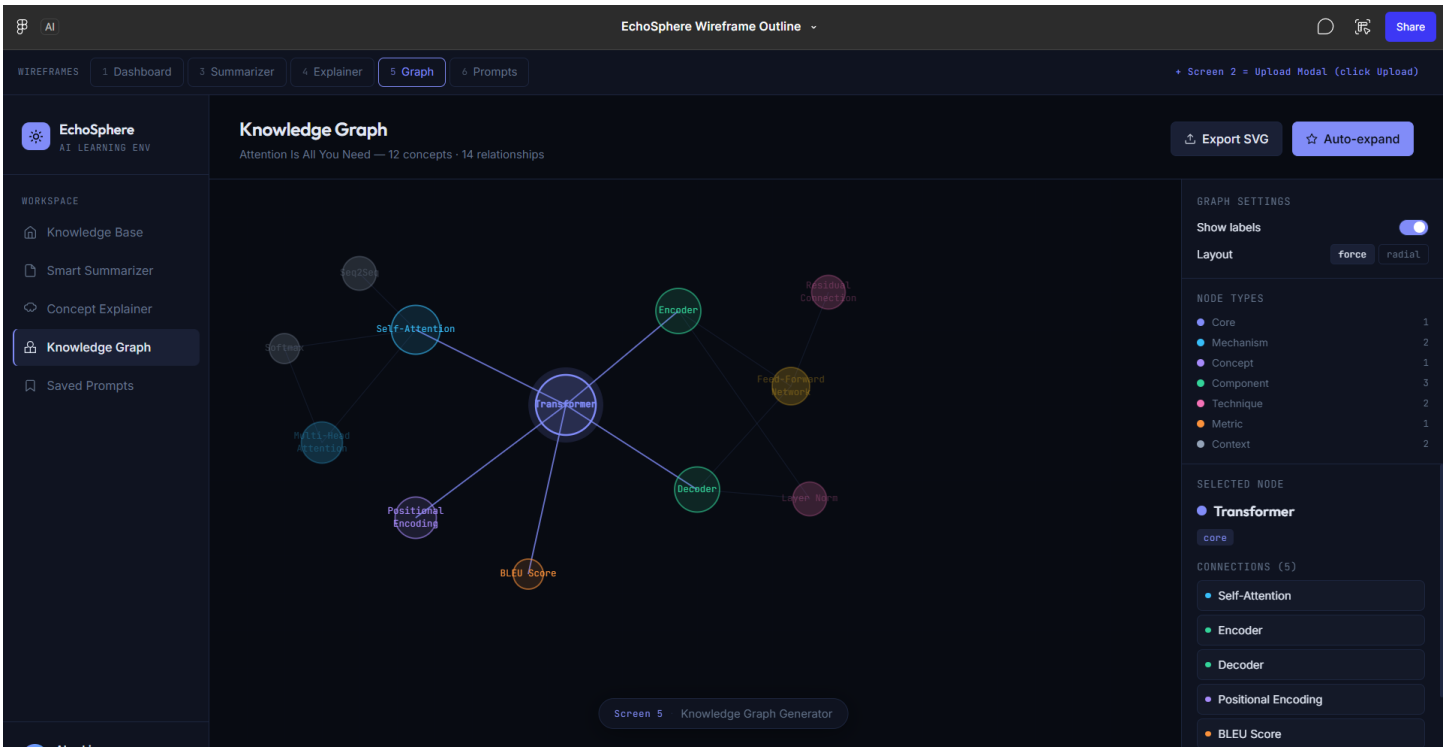
LINK - https://www.figma.com/make/o0oFJ9qgmdbQpxuekEYeyl/EchoSphere-Wireframe-Outline?fullscreen=1&t=5TR1E21N2il7zOKA-1&code-node-id=1-5

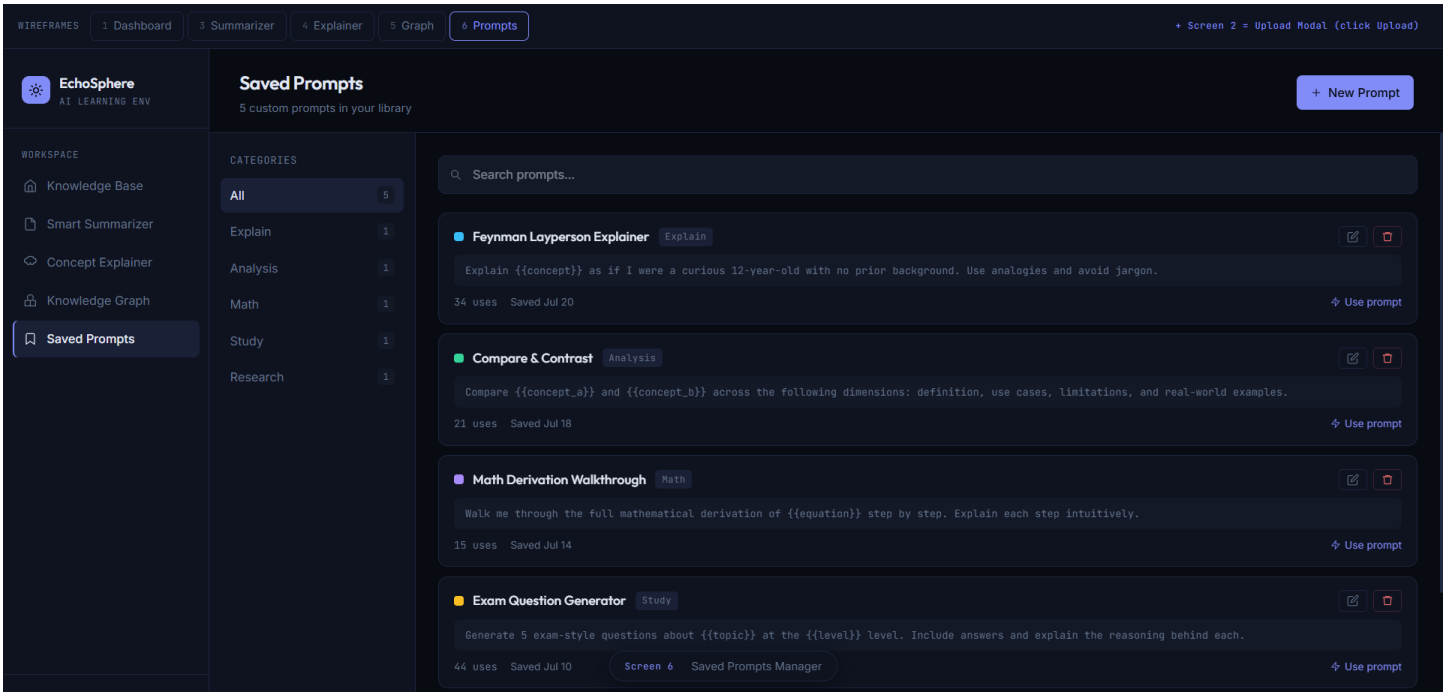



Upload Documents
Supported formats: PDF, DOCX, TXT, MD
Drag & drop files here
or browse your computer
PDF
DOCX
TXT
MD
SAVE TO COLLECTION
Machine Learning
Cancel
Upload Files
Screen 1
Dashboard / Knowledge Manager

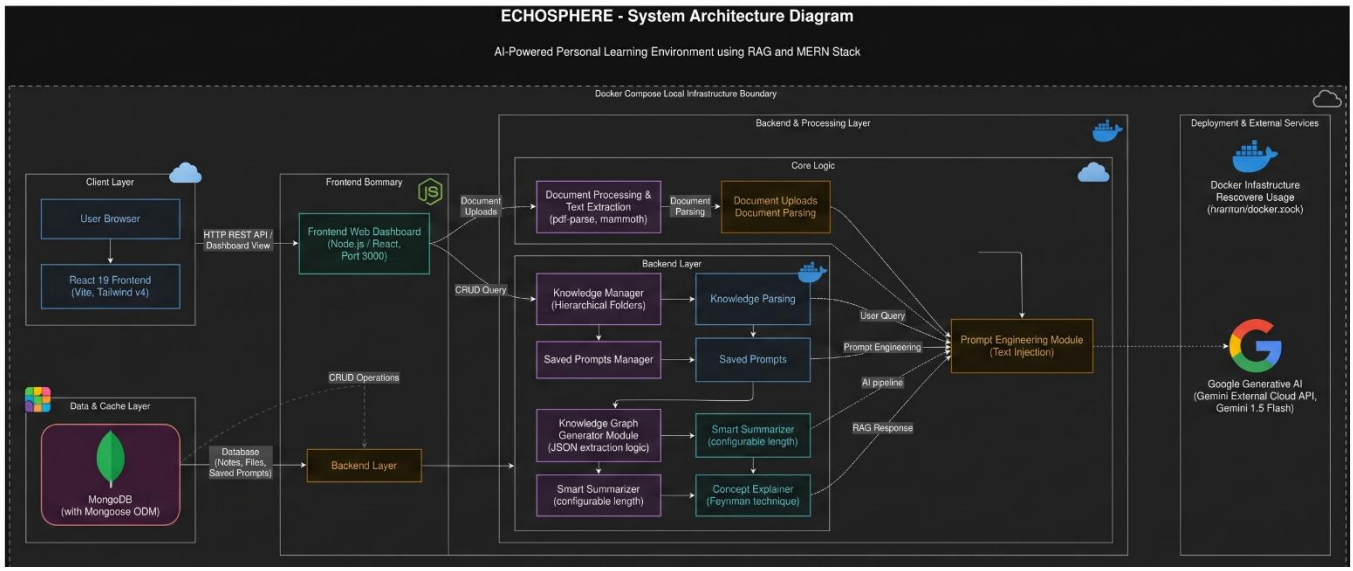

AI
EchoSphere Wireframe Outline
Share
WIREFRAMES
1 Dashboard
3 Summarizer
4 Explainer
5 Graph
6 Prompts
+ Screen 2 = Upload Modal (click Upload)
EchoSphere
AI LEARNING ENV
Smart Summarizer
Attention Is All You Need.pdf — Machine Learning
AI Summary
Export
WORKSPACE
Knowledge Base
Smart Summarizer
Concept Explainer
Knowledge Graph
Saved Prompts
DOCUMENT VIEWER
1 / 12
AI SUMMARY
Brief
Standard
Detailed
Attention Is All You Need
Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit
Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin
Google Brain · Google Research · University of Toronto
ABSTRACT
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely.
Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU.
1. INTRODUCTION
Recurrent neural networks, long short-term memory and gated recurrent neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation...
Pages 12
Words 9,843
Read Time ~37 min
Year 2017
KEY CONCEPTS
Multi-Head Attention
Positional Encoding
Transformer
Self-Attention
BLEU Score
Encoder-Decoder
SUMMARY — STANDARD
The Transformer architecture, proposed by Vaswani et al. (2017), fundamentally departs from recurrent and convolutional networks by relying entirely on attention mechanisms for sequence transduction tasks.
The model employs multi-head self-attention, which allows the model to jointly attend to information from different representation subspaces at different positions. Positional encodings inject sequence order information since the architecture contains no recurrence.
Key results include 28.4 BLEU on EN→DE translation and 41.0 BLEU on EN→FR. Training required 3.5 days on 8 P100 GPUs — significantly less than competing models. The model also generalizes well to English constituency parsing.
Explain Concepts
Build Graph
Save
Screen 3
Smart Summarizer Workspace


AI
EchoSphere Wireframe Outline
Share
WIREFRAMES
1 Dashboard
3 Summarizer
4 Explainer
5 Graph
6 Prompts
+ Screen 2 = Upload Modal (click Upload)
EchoSphere
AI LEARNING ENV
WORKSPACE
Knowledge Base
Smart Summarizer
Concept Explainer
Knowledge Graph
Saved Prompts
Concept Explainer
Feynman Technique — understand anything at any depth
CONCEPT OR TOPIC
Self-Attention mechanism in Transformers
EXPLANATION DEPTH
Layperson
No prior knowledge
Beginner
Basic concepts only
Intermediate
Some ML background
Advanced
Technical proficiency
Expert
Research-level
GROUND IN DOCUMENT (optional)
Attention Is All You Need.pdf
INTERMEDIATE LEVEL · For someone with basic ML knowledge
Self-Attention mechanism in Transformers
Self-Attention is a mechanism that computes a weighted representation of a sequence by allowing each position to attend to all other positions. Given input sequence X, it produces queries Q, keys K, and values V via learned linear projections.
The attention weights are computed as:
Attention(Q, K, V) = softmax(QK^T / √d_k) · V
The scaling factor √d_k prevents the dot products from growing too large in high-dimensional spaces, which would push softmax into regions with very small gradients.
EXPLORE FURTHER
How does multi-head attention work?
What is positional encoding?
Compare RNNs vs Transformers
What are attention weights?
Screen 4 Concept Explainer (Feynman)

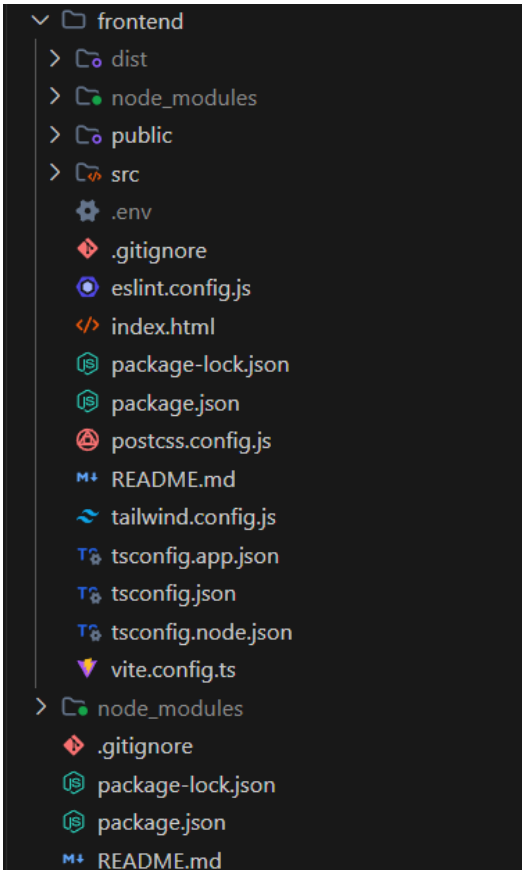

AI
EchoSphere Wireframe Outline
Share
WIREFRAMES
1 Dashboard
3 Summarizer
4 Explainer
5 Graph
6 Prompts
+ Screen 2 = Upload Modal (click Upload)
EchoSphere
AI LEARNING ENV
WORKSPACE
Knowledge Base
Smart Summarizer
Concept Explainer
Knowledge Graph
Saved Prompts
Knowledge Graph
Attention Is All You Need — 12 concepts · 14 relationships
Export SVG
Auto-expand
Seq2Seq
Self-Attention
Softmax
Multi-Head Attention
Transformer
Encoder
Residual Connection
Feed-Forward Network
Decoder
Layer Norm
Positional Encoding
BLEU Score
GRAPH SETTINGS
Show labels
Layout
force
radial
NODE TYPES
Core 1
Mechanism 2
Concept 1
Component 3
Technique 2
Metric 1
Context 2
SELECTED NODE
Transformer
core
CONNECTIONS (5)
Self-Attention
Encoder
Decoder
Positional Encoding
BLEU Score
Screen 5 Knowledge Graph Generator

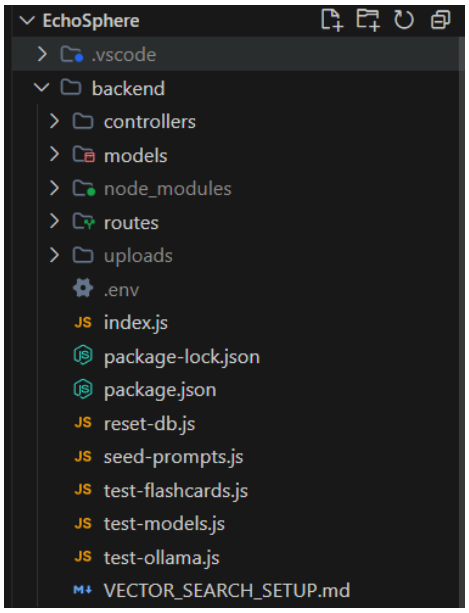


4. Architecture Diagram



Folder structure:



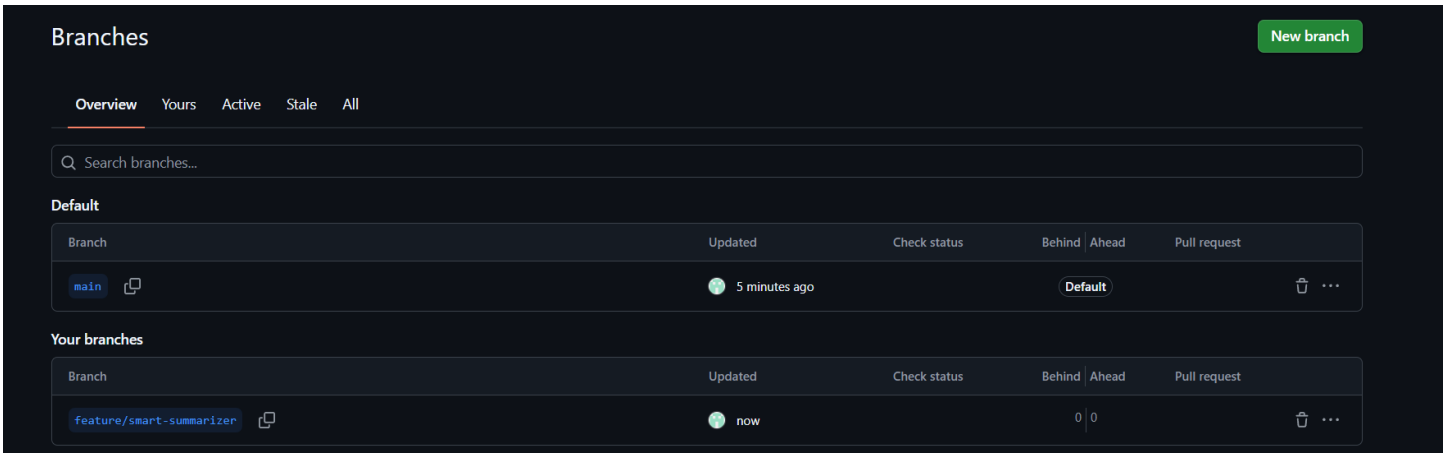


Branching Strategy:

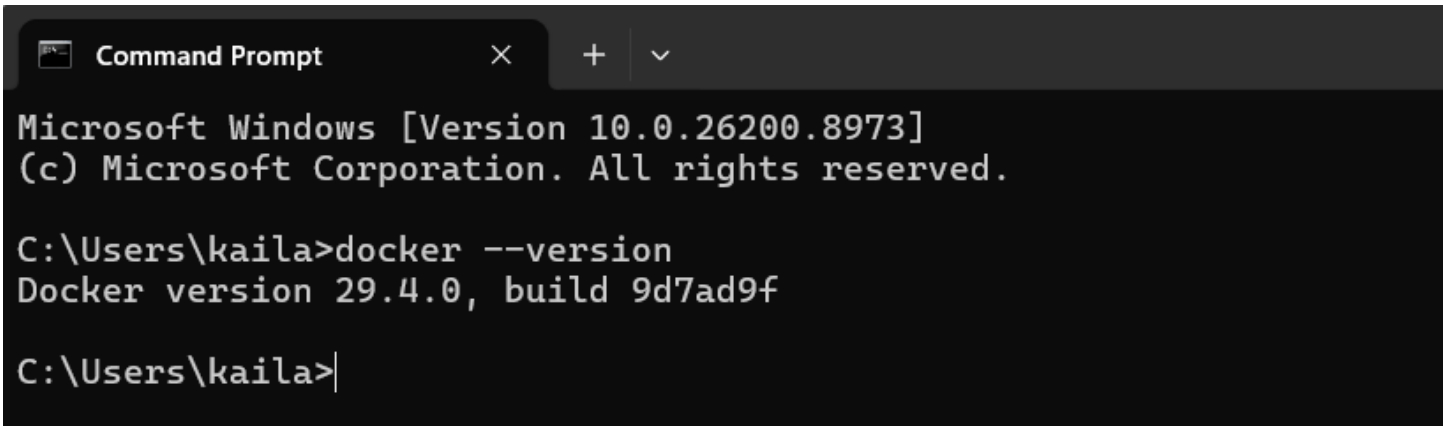
Branching Strategy (GitHub Flow)

For EchoSphere, we strictly adhere to the **GitHub Flow** branching strategy to maintain a clean, organized, and stable repository during development.

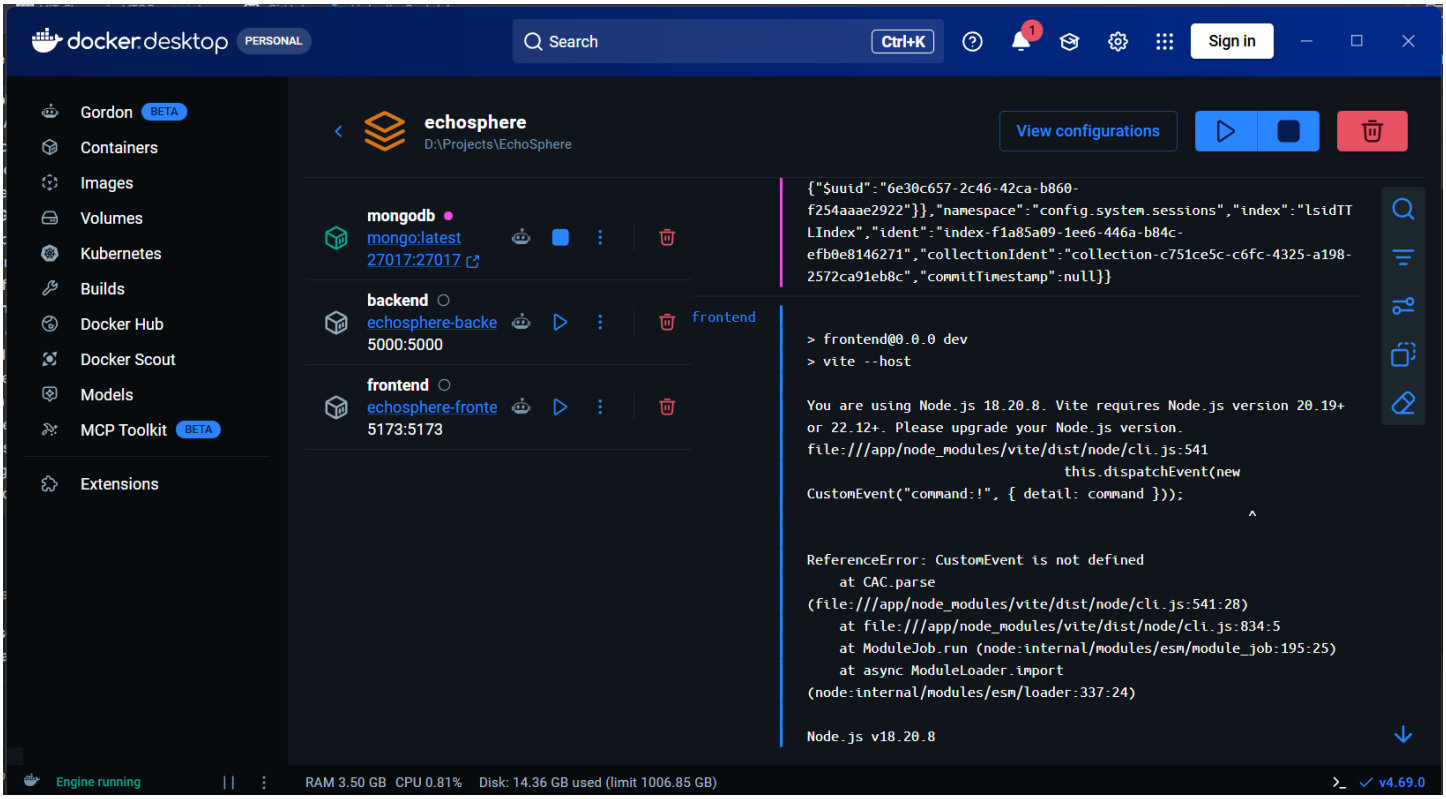
1. `main` **Branch:** This is the primary, production-ready branch. All code in `main` is stable and functional. Direct commits to `main` are avoided to prevent breaking the core application.
2. **Feature Branches:** For every new module (e.g., Knowledge Graph, Concept Explainer), a dedicated branch is created from `main`. We use descriptive naming conventions such as:
   - `feature/pdf-ingestion`
   - `feature/knowledge-graph-ui`
   - `bugfix/gemini-api-key-error`
3. **Pull Requests (PRs):** Once a feature is complete locally, a Pull Request is opened against the `main` branch. This allows for code review and automated checks before integration.
4. **Merge & Delete:** After a successful review and merge into `main`, the feature branch is deleted to keep the repository uncluttered.



5. Dev setup proof Docker Desktop installed



Successful docker build and docker run / docker-compose up



```
D:\Projects\EchoSphere>docker-compose up --build
time="2026-07-30T23:16:38+05:30" level=warning msg="D:\\Projects\\EchoSphere\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, p
lease remove it to avoid potential confusion"
[+] up 9/9
 ✔Image mongo:latest Pulled                                                                                    64.6s
#1 [internal] load local bake definitions
#1 reading from stdin 977B done
#1 DONE 0.0s

#2 [frontend internal] load build definition from Dockerfile
#2 transferring dockerfile: 425B 0.0s done
#2 DONE 0.1s

#3 [backend internal] load build definition from Dockerfile
#3 transferring dockerfile: 360B 0.0s done
#3 DONE 0.1s

#4 [frontend internal] load metadata for docker.io/library/node:18-alpine
#4 DONE 3.2s

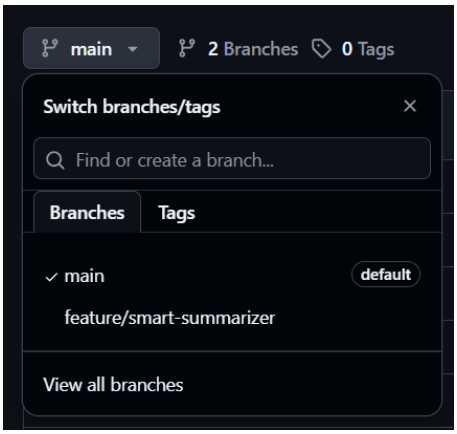
#5 [frontend internal] load .dockerignore
#5 transferring context: 2B done
#5 DONE 0.1s
```

```
#18 [frontend 5/5] COPY . .
#18 DONE 4.3s

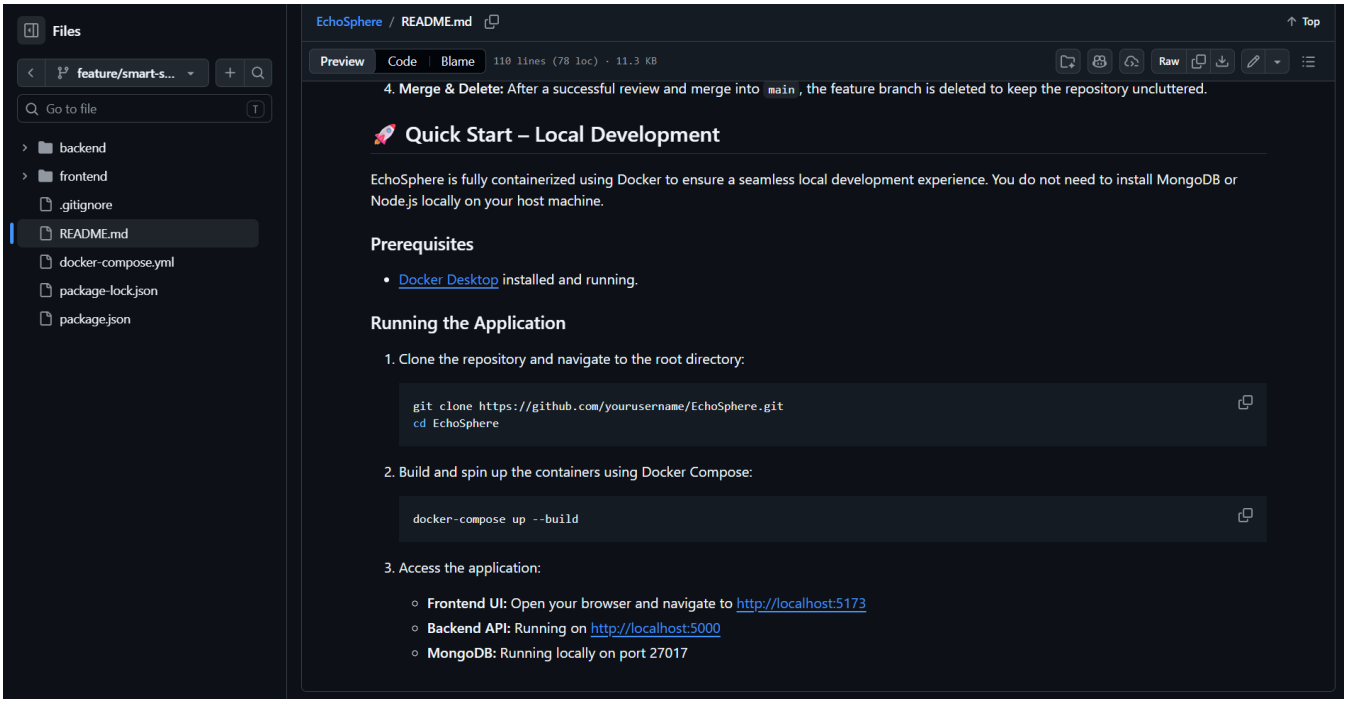
#19 [frontend] exporting to image
#19 exporting layers
#19 exporting layers 2.1s done
#19 writing image sha256:dc3c04ef142ecbc598836cd5266544dba80d381445efdec65103190900213 2ac done
#19 naming to docker.io/library/echosphere-frontend done
#19 DONE 2.1s

#20 [frontend] resolving provenance for metadata file
#20 DONE 0.0s
[+] up 7/7
 ✔Image echosphere-frontend       Built                                                                        74.3s
 ✔Image echosphere-backend        Built                                                                        74.3s
 ✔Network echosphere_default      Created                                                                      0.1s
 ✔Volume echosphere_mongo_data    Created                                                                      0.0s
 ✔Container echosphere-mongodb-1  Created                                                                      0.1s
 ✔Container echosphere-backend-1  Created                                                                      0.1s
 ✔Container echosphere-frontend-1 Created                                                                      0.1s
Attaching to backend-1, frontend-1, mongodb-1
mongodb-1  | {"t":{"$date":"2026-07-30T17:50:44.691+00:00"},"s":"I",  "c":"-",        "id":8991200, "ctx":"main","msg":"Shuffling initializers","attr":{"see
d":4002069357}}
```

GitHub repo page showing branches (main + feature branch)



GitHub repo homepage showing the rendered README with Quick Start



PROJECT REPO (GITHUB) :

